

Visibility-Sensitive Command Streaming for Native GGML FPGA Inference

Method. Visibility-Sensitive Trace Compression (VSTC)

Motivation

The bottleneck is a control-plane mismatch in native, low-batch FPGA inference. A GGML trace can include calls whose results must stay visible to the host or take part in mutable state, alongside a larger device-side bulk whose intermediate completions do not need to be surfaced. Treating both groups as equally observable turns a semantic safeguard into repeated XRT launch traffic. The main example is llama.cpp running Gemma 3 1B F16 on a U280, but the issue is structural for native tensor runtimes whose call graph and buffer identities are only known at execution time.

Recent FPGA LLM work makes the remaining control-plane gap easier to see. Big PE [openalex:W4416342314] and XtraMAC [arxiv:2605.06052v1] both push the compute path toward higher density or better datatype use, so host dispatch becomes a larger share of low-batch latency instead of a minor overhead. At the same time, llama.cpp exposes a stable native GGML call stream, XRT exposes the command and completion interfaces needed to capture it, and U280-class boards can host a small resident interpreter without replacing the installed F16 kernels. These conditions make it practical to test a runtime-representation change directly rather than infer its value from a model-specific accelerator redesign.

Prior work stops short for a structural reason. Big PE [openalex:W4416342314] changes the compute array mapping, which has no runtime record of host reads, aliases, state, or consumer visibility. LlamaF [openalex:W4405909140] uses model-specific matrix grouping and keeps attention control and KV-cache work on the processing system, but it does not build patchable exact templates from the live GGML trace or give each original call an individually surfaced completion after eligibility is decided. XtraMAC [arxiv:2605.06052v1] improves MAC utilization, but a MAC microarchitecture cannot decide whether a runtime call may defer host-visible materialization without changing native behavior.

If this gap closes, a native llama.cpp backend can keep its original tensor allocation and explicit-error contract while avoiding per-call host launches for the device-only parts of prefill and decode. That gives one reusable runtime path across traces instead of requiring each new model to be recast into a bespoke matrix schedule, and it makes the latency claim easier to attribute to a measured reduction in unnecessary submissions rather than to a bundled accelerator redesign.

Method

ticketing_and_partition

Record what each native call exposes, then split the trace into host-visible boundaries and safe device-only bulk.

1. Add a tracing layer around the GGML tensor runtime calls and Xilinx Runtime (XRT) submission/completion events. Give every original call a stable $call_{id}$ and record its operator name, tensor handles, XRT buffer object id, byte offset and length, command queue id, dependency events, completion event, return code, and any explicit error message. Build the per-call ticket by checking seven concrete facts: whether the host can read the result at that point, whether live tensor views share overlapping bytes, whether the call mutates shared runtime or queue state, whether there is exactly one later device-side consumer before any host-visible boundary, whether the output remains in the caller's original allocation, whether temporary scratch storage has externally shared lifetime or address, and whether the original call exposes an error at that boundary. Write the ordered ticket trace as JSON Lines, one record per $call_{id}$, in the same order as the native execution. *Visibility-sensitivity ticket for original GGML call i : host-read, alias, state, unique-consumer, original-allocation, scratch, and explicit-error facts.*

$$\tau_i = (r_i, a_i, s_i, u_i, o_i, q_i, e_i) \quad (1)$$

Why: The compression decision has to come from what the running program can observe, not from the operator name alone or from a static model group.

2. Run a deterministic partition pass over the ticket trace. For each $call_{id}$, output a decision record with the $call_{id}$, whether it is a host-visible boundary (H) or a bulk device-only entry (B), the exact denied reasons, and its predecessor and successor call ids. Put a call in H if the ticket shows any host read, byte overlap with another live tensor view, shared state mutation, more than one device consumer or no unique consumer, changed caller allocation, externally shared scratch storage, or an explicit error visible at that call boundary. Put a call in B only if all of those hazards are absent and the unique-consumer and original-allocation facts are true. Use the dynamic

tensor-use graph and XRT event dependencies to group consecutive B calls into maximal runs, separated by H calls, and save machine-readable denied reasons for every call that was not admitted. *Admission predicate: only a call with no host-read, alias, state, scratch, or explicit-error hazard and with a unique consumer and original allocation can enter a B template.*

$$b_i = \mathbb{K}[\neg r_i \wedge \neg a_i \wedge \neg s_i \wedge u_i \wedge o_i \wedge \neg q_i \wedge \neg e_i] \quad (2)$$

Why: The partition must not cross the visibility, alias, state, allocation, scratch, or error conditions that define the native contract.

template_streaming

Turn eligible bulk into exact streamed templates and run them while keeping boundary behavior visible.

- For each consecutive B run, create a B-template record instead of inventing a new fused operator. The template header should contain *atemplate_{id}*, the ordered *call_{ids}*, the existing 16-bit floating point (F16) kernel identifiers, an argument-slot table, XRT buffer object bindings, byte offsets, tensor shapes, strides, element types, dependency-event slots, completion slots, and patch locations for values that change at runtime. The bytecode body is an ordered list of the same kernel invocations and XRT command descriptors that the native backend would have issued, but with symbolic slots filled from the current GGML tensor bindings just before execution. Keep one completion slot for every original *call_{id}* so the interpreter can report call-level completion in native order. If a binding cannot be expressed as a stable buffer object plus byte offset and length, reject that call from the template and record *atemplate_build_denial* reason.

Why: Exact templates keep the installed F16 kernels and caller allocation semantics while removing repeated host-side reconstruction of device-only submissions.

- Run the mixed H/B stream with a resident interpreter that keeps a template cache and an in-order completion ledger keyed by original *call_{id}*. When the next item is H, call the original GGML/XRT path unchanged, expose completion exactly where the native backend would expose it, and copy the same return code and explicit error payload. Before crossing that H boundary, make visible any earlier B completions that the native order requires. When the next item is a B template, fill its symbolic argument slots from the current tensor bindings, enqueue its encoded command sequence to the resident interpreter, run the existing kernels in the recorded order, and update the completion ledger for each original *call_{id}*. Do not expose intermediate B completions to the host unless a later H boundary requires them. Return logits, tensor side effects, completion states, and explicit-error traces in native *call_{id}* order, and map any interpreter internal failure back to the earliest original call affected.

Why: This gives the claimed launch reduction without turning the resident engine into a generic asynchronous scheduler that hides native runtime observability.

equivalence_and_measurement

Check that the streamed path still matches native outputs and measure whether admission actually helps.

- Run three matched versions of the same test cases: the native backend, VSTC with its admitted H/B stream, and VSTC with every possible B entry forced back to individual completion while keeping the resident interpreter path otherwise the same. Use the same prompts, model weights, tensor allocations, and XRT configuration for all three. Check final logits byte for byte and compare explicit-error traces as ordered records containing *call_{id}*, return code, and error payload; report the first *call_{id}* where they differ. Compute the admission fraction ρ from the saved H/B decisions as the share of baseline XRT submissions represented by admitted B-template entries. Also count actual XRT submissions and report median (p50) and tail (p95) latency for prefill and decode from timestamped run logs, excluding warmup runs by a fixed rule chosen before measurement. As a full-observation check, rerun with all host-read facts forced true; this should remove B admission and reproduce the native completion and error trace. The forced-denial run shows whether the latency change follows the admitted fraction rather than just the resident interpreter path. *Admission fraction rho, the share of baseline XRT submissions represented by admitted B-template entries.*

$$\rho = \frac{\sum_{i=1}^{N_{\text{XRT}}} b_i}{N_{\text{XRT}}} \quad (3)$$

Why: The full-observation oracle and forced B-denial control test whether any speedup comes from visibility-sensitive admission rather than from an unmeasured interpreter or a changed kernel path.